

| Semester       | Jahr | Abgabedatum | Vortragsdatum |
|----------------|------|-------------|---------------|
| Sommersemester | 26   | 30.07.2026  | 31.07.2026    |

**Vorlesung:** Innovative Konzepte zur Programmierung von Industrierobotern **Dozent:** Prof. Dr.-Ing. Björn Hein

Projektaufgabe - SBL: bidirektionale Planung mit Lazy Collision Checking

Ziel der Projektaufgabe ist die Implementierung und Evaluation eines SBL-inspirierten Planers. SBL steht für Single-query, Bidirectional, Lazy Collision Checking. Im Gegensatz zu PRM-basierten Multi-Query- Ansätzen wird gezielt ein einzelnes Start-Ziel-Problem gelöst. Dazu wachsen zwei Bäume, einer vom Start und einer vom Ziel. Kollisionsprüfungen werden möglichst lange verzögert und erst dann durchgeführt, wenn eine potenzielle Verbindung zwischen beiden Bäumen gefunden wurde.

Diese Aufgabe ist für eine Gruppe mit drei Personen ausgelegt. Sie verbindet drei zentrale Konzepte: bidirektionales Baumwachstum, Lazy Evaluation und adaptive lokale Kollisionsprüfung.

Die zentrale Abgabe ist ein Jupyter Notebook. Dieses Notebook dient als technischer Bericht: Es soll Vorgehensweise, verwendete Module, Experimente, Visualisierungen, Ergebnisse und Diskussion nachvollziehbar enthalten.

Der eigentliche Python-Code soll nach Möglichkeit in Modulen implementiert und im Notebook importiert werden. Das Notebook soll nicht aus langen Codeblöcken bestehen, sondern die Lösung strukturiert erklären, ausführen und auswerten.

Ausgangspunkt sind insbesondere:

- notebooks/IPRRT.py
- notebooks/IPVISRRT.py
- notebooks/IPLazyPRM.py
- notebooks/IPVISLazyPRM.py
- notebooks/IPEnvironment.py
- notebooks/IPEnvironmentKin.py
- notebooks/IPPlanarManipulator.py
- notebooks/IPPerfMonitor.py
- notebooks/IP-7-0-PRM-Lazy.ipynb
- notebooks/IP-9-0-RRT-Basics.ipynb
- notebooks/IP-10-0-PlanarManipulator.ipynb
- notebooks/IP-X-0-Benchmarking-concept.ipynb
- notebooks/IP-X-1-Automated\_PlanerTest.ipynb

## I.) Erwartete Notebook-Struktur

### 1. Einleitung und Zielsetzung

Beschreiben Sie,

1. was unter einem Single-Query-Planer verstanden wird,
2. warum bidirektionales Baumwachstum hilfreich sein kann,
3. was Lazy Collision Checking bedeutet,
4. warum lokale Verbindungstests oft der teure Teil der Planung sind,
5. welche Fragestellungen Sie experimentell untersuchen.

### 2. Analyse vorhandener Planer

Analysieren Sie die vorhandenen Module:



1. IPRRT.py als Beispiel für baumbasiertes Wachstum,
2. IPLazyPRM.py als Beispiel für verzögerte Kantenprüfung,
3. IPEnvironment.py und IPEnvironmentKin.py für Punkt- und Linientests,
4. IPPerfMonitor.py für Messung von Aufrufen und Laufzeiten.

Dokumentieren Sie, welche Konzepte übernommen werden können und welche Teile für einen SBL-inspirierten Planer neu implementiert werden müssen.

### 3. Grundkonzept des SBL-inspirierten Planers

Entwickeln Sie ein Konzept mit zwei Bäumen:

1. Startbaum  $T_{start}$ ,
2. Zielbaum  $T_{goal}$ ,
3. zufälliges Sampling im Konfigurationsraum,
4. Erweiterung eines Baums in Richtung eines Samples,
5. regelmäßiger Verbindungsversuch zum anderen Baum,
6. lazy Validierung einer potenziellen Gesamtlösung.

Der Planer soll für n-DoF-Konfigurationen funktionieren.

### 4. Datenstruktur für Bäume und Kantenstatus

Implementieren Sie eine geeignete Graphstruktur.

Anforderungen:

1. Knoten speichern Konfigurationen,
2. Kanten speichern Eltern-Kind-Beziehungen,
3. Kanten besitzen einen Status, z.B. **unknown**, **valid** oder **invalid**,
4. die beiden Bäume müssen getrennt oder eindeutig markiert verwaltet werden,
5. eine potenzielle Lösung muss aus beiden Bäumen zusammengesetzt werden können.

### 5. Baumwachstum

Implementieren Sie eine Erweiterungsstrategie.

Mindestens erwartet:

1. zufälliges freies Sample,
2. Auswahl eines nächsten Knotens im aktiven Baum,
3. Erweiterung mit Schrittweite  $\epsilon$ ,
4. optionaler **Goal Bias** oder **Tree Bias**, 
5. Einfügen der neuen Kante zunächst als **unknown**.

Diskutieren Sie, ob eine Kante bereits beim Einfügen vollständig geprüft werden soll oder ob sie zunächst ungeprüft bleiben kann.

### 6. Verbindungsversuch zwischen den Bäumen

Implementieren Sie eine Strategie, um die beiden Bäume zu verbinden.

Mögliche Varianten:

1. nächster Knoten im anderen Baum,
2. mehrere nahe Knoten,
3. periodischer Verbindungsversuch nach einer festen Anzahl Erweiterungen,
4. Verbindungsversuch nur bei räumlicher Nähe.

Potenzielle Verbindungskanten sollen zunächst ebenfalls als **unknown** gespeichert werden.

## 7. Lazy Validierung einer potenziellen Lösung

Wenn eine Verbindung zwischen Start- und Zielbaum gefunden wurde, entsteht ein Kandidatenpfad.

Validieren Sie diesen Pfad lazy:

1. Prüfen Sie nur Kanten, die für den Kandidatenpfad benötigt werden.
2. Bereits geprüfte gültige Kanten sollen wiederverwendet werden.
3. Wird eine Kante als kollidierend erkannt, markieren Sie sie als **invalid**.
4. Entfernen oder ignorieren Sie ungültige Kanten bei weiteren Kandidatenpfaden.
5. Suchen Sie danach weiter, bis ein vollständig gültiger Pfad gefunden wird oder das Iterationslimit erreicht ist.

## 8. Adaptive lokale Kollisionsprüfung

Implementieren Sie für die Validierung von Kanten eine adaptive Mittelpunktprüfung.

Anforderungen:

1. Zuerst wird der Mittelpunkt der Verbindung geprüft.
2. Danach werden rekursiv bzw. iterativ die Mittelpunkte der Teilintervalle geprüft.
3. Die Unterteilung endet, wenn die Segmentlänge kleiner als **epsilon** ist.
4. Start- und Endpunktprüfung soll konfigurierbar sein.
5. Der Test soll sofort abbrechen, sobald eine Kollision gefunden wird.
6. Die Reihenfolge der getesteten Punkte soll dokumentiert und visualisierbar sein.

Vergleichen Sie diesen Test mit einem gleichmäßig diskretisierten Linientest.



## 9. Visualisierung

Erweitern Sie die Visualisierung so, dass die Arbeitsweise des Planers nachvollziehbar wird.

Visualisieren Sie mindestens:

1. Startbaum,
2. Zielbaum,
3. ungeprüfte Kanten,
4. gültig geprüfte Kanten,
5. ungültige Kanten,
6. Kandidatenpfad,
7. finalen gültigen Pfad,
8. getestete Punkte einer adaptiven Kantenprüfung.

## 10. Evaluation mit 2-DoF-Punktrobotern

Evaluieren Sie den Planer anhand von mindestens drei 2-DoF-Benchmarkumgebungen.

Die Umgebungen sollen unterschiedliche Eigenschaften besitzen:

1. offene Umgebung,
2. Engstelle,
3. fallenartige oder stark zerklüftete Umgebung.

Führen Sie pro Benchmark und Parameterkombination mindestens 30 Planungsläufe durch.

#### 11. Evaluation mit n-DoF-PlanarManipulator

Evaluieren Sie den Planer zusätzlich anhand von mindestens zwei PlanarManipulator-Beispielen.

Betrachten Sie mindestens:

1. 2-DoF-PlanarManipulator,
2. 4-DoF-PlanarManipulator,
3. optional 6-DoF-PlanarManipulator als Vertiefung.

Nutzen Sie die Beispiele aus `IP-10-0-PlanarManipulator.ipynb` und `IPEnvironmentKin.py`.

#### 12. Vergleich mit anderen Planern

Vergleichen Sie den SBL-inspirierten Planer mit mindestens zwei vorhandenen Verfahren.

Sinnvolle Vergleiche sind:

1. RRT,
2. LazyPRM,
3. BasicPRM,
4. RRT\* als optionaler Zusatzvergleich.

Der Vergleich soll nicht nur die Pfadlänge betrachten, sondern auch die Anzahl und Art der Kollisionsprüfungen.

#### 13. Statistik und Parameterstudie

Erfassen Sie mindestens:

1. Erfolgsrate,
2. Planungszeit,
3. Zeit bis zum ersten Kandidatenpfad,
4. Zeit bis zum ersten gültigen Pfad,
5. Anzahl erzeugter Knoten pro Baum,
6. Anzahl ungeprüfter, gültiger und ungültiger Kanten,
7. Anzahl Punktkollisionstests,
8. Anzahl Linientests,
9. Anzahl abgebrochener adaptiver Tests,
10. Länge des finalen Pfads,
11. Anzahl Kandidatenpfade vor der gültigen Lösung.

Variieren Sie mindestens:

1. Schrittweite `eta`,
2. adaptive Auflösung `epsilon`,
3. Häufigkeit der Verbindungsversuche,
4. Sampling- oder Goal-Bias-Strategie.

#### 14. Validierung der Korrektheit

Prüfen Sie Ihre Implementierung gezielt.

Mindestens erwartet:

1. Unit-Test für den adaptiven Linientest, ✓
2. Test, dass bekannte gültige Kanten nicht erneut geprüft werden müssen, ✓

3. Test, dass ungültige Kanten nicht weiter für Kandidatenpfade verwendet werden,
4. Test, dass ein zusammengesetzter Pfad korrekt vom Start zum Ziel führt, ✓
5. Prüfung, dass alle Kanten des finalen Pfads kollisionsfrei sind. ✓

#### 15. Arbeitsteilung und Integrationskonzept

Dokumentieren Sie kurz, wie die Arbeit in der Gruppe aufgeteilt wurde. Eine sinnvolle Aufteilung ist z.B.:

1. Baumwachstum, Verbindungsversuche und Graphdatenstruktur,
2. Lazy Validierung, adaptive Kollisionsprüfung und Tests,
3. Visualisierung, Benchmarks und statistische Auswertung.

Die Teilmodule sollen am Ende in einem gemeinsamen Notebook zusammengeführt werden.

#### 16. Diskussion und Fazit

Beantworten Sie:

1. Wann hilft bidirektionales Baumwachstum?
2. Wann spart Lazy Collision Checking tatsächlich Aufwand?
3. In welchen Fällen erzeugt Lazy Collision Checking zusätzlichen Verwaltungsaufwand?
4. Wann ist die adaptive Mittelpunktprüfung vorteilhaft?
5. Wie unterscheidet sich der Planer praktisch von RRT und LazyPRM?
6. Welche Parameter sind besonders kritisch?
7. Welche Variante würden Sie für weitere Experimente weiterentwickeln?

#### II.) Präsentation

Zusätzlich zur Notebook-Abgabe gibt es eine Präsentation. Sie können dafür entweder das Notebook direkt als Präsentationsgrundlage verwenden oder daraus separate Folien erzeugen.

Der Vortragstag ist der 31.07.2026.

Die Präsentation soll nicht den gesamten Code erklären, sondern die wichtigsten Entscheidungen, Visualisierungen, Messergebnisse und Erkenntnisse herausarbeiten.

#### III.) Verwendung von KI-Werkzeugen

Sie dürfen Codex oder andere KI-Werkzeuge verwenden.

Die Bewertung erfolgt nicht danach, ob Code vollständig ohne Hilfsmittel geschrieben wurde, sondern danach, ob Sie die Lösung verstehen, integrieren, testen und kritisch bewerten können.

Dokumentieren Sie kurz:

1. wofür KI verwendet wurde,
2. welche Vorschläge übernommen wurden,
3. welche Vorschläge verworfen wurden,
4. wie Sie geprüft haben, dass der erzeugte Code korrekt funktioniert.

#### IV.) Abgabe

Abzugeben sind:

1. Jupyter Notebook als technischer Bericht,
2. Python-Module mit dem SBL-inspirierten Planer,
3. adaptive Kollisionsprüfung bzw. lokale Validierung,
4. angepasste oder ergänzte Visualisierungsmodule,

5. Tests für Baumstruktur, Lazy Validierung und Kollisionsprüfung,
6. Benchmarkauswertungen,
7. erzeugte Abbildungen/Animationen, sofern sie nicht direkt im Notebook enthalten sind,
8. optional separate Präsentationsfolien.

Anmerkung: Bitte checken Sie die Notebooks „IP-X-0-Benchmarking-concept.ipynb“ und „IP-X-1-Automated\_PlanerTest.ipynb“ für Profiling und Statistiken.

Viel Erfolg!